ТОО «Колледж Хекслет»

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ

на тему: «Разработка веб-приложения на тему «Конструктор персональной «Витрины ссылок» (Linktree-аналог с кастомизацией)»

по модулю: «Разработка модулей программного обеспечения для компьютерных систем»
Специальность: 06130100 – Программное обеспечение (по видам)
Квалификация: 4S06130105 – Техник информационных систем

Выполнил: студент 3 курса
8 ТИС/23 группы
Есқали Бейбарыс Жеңісұлы

(буква) (циф.экв.) (балл)
Допущен к защите «___»_____20__г.
Преподаватель: _____ **Турман.Н.Р**
(подпись)

Защитил с оценкой:
____ _____ _____
(буква) (циф.экв.) (балл)

Алматы 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....2
1 ПОСТАНОВКА ЗАДАЧИ (ТЕХНИЧЕСКОЕ ЗАДАНИЕ).....4
1.1 Цель проекта.....4
1.2 Анализ существующих аналогов.....4
1.3 Функциональные требования.....6
1.4 Нефункциональные требования.....8
1.5 Критерии готовности.....9
2 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ.....12
2.1 Выбор технологического стека и его обоснование.....12
2.2 Проектирование структуры данных.....13
2.4 Внешняя структура приложения.....16
2.5 Внутренняя структура приложения.....17
3 ПРАКТИЧЕСКАЯ ЧАСТЬ РЕАЛИЗАЦИИ ПРОЕКТА.....19
3.1 Настройка проекта и рабочего окружения.....19
3.2 Разработка конфигурационного файла данных.....20
3.3 Разработка корневого компонента и глобальных стилей.....20
3.4 Разработка компонента фонового эффекта.....21
3.5 Разработка компонента шапки профиля.....22
3.6 Разработка компонента карточки ссылки.....23
3.7 Разработка компонента списка ссылок.....24
3.8 Интеграция компонентов и передача данных.....25
3.9 Обеспечение адаптивности интерфейса.....26
3.10 Тестирование приложения.....26
3.11 Сборка и подготовка к развёртыванию.....27
ЗАКЛЮЧЕНИЕ.....28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....29

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и повсеместного распространения социальных сетей персональное онлайн-присутствие стало неотъемлемой частью профессиональной и творческой деятельности. Специалисты различных областей — разработчики, дизайнеры, фотографы, музыканты, блогеры и предприниматели — нуждаются в едином инструменте, позволяющем представить все свои ресурсы в интернете в виде одной компактной и визуально привлекательной страницы. Именно эту задачу решают сервисы класса «link-in-bio», наиболее известным представителем которых является платформа Linktree.

Вместе с тем существующие коммерческие решения обладают рядом существенных недостатков. Во-первых, расширенные возможности кастомизации внешнего вида в большинстве подобных сервисов доступны только по платной подписке. Во-вторых, пользователь ограничен предложенными шаблонами и не имеет возможности реализовать уникальный дизайн, отражающий его индивидуальность. В-третьих, данные пользователя хранятся на сторонних серверах, что порождает зависимость от политики конкретного сервиса. Разработка собственного аналога подобного инструмента позволяет устранить все перечисленные недостатки и одновременно представляет собой актуальную и практически значимую учебную задачу.

В рамках учебной практики по профессиональному модулю ПМ3 «Разработка модулей программного обеспечения для компьютерных систем» авторами проекта было разработано веб-приложение «Link Showcase» — персональный конструктор витрины ссылок с поддержкой визуальной кастомизации. Приложение реализовано с применением прогрессивного JavaScript-фреймворка Vue 3 актуальной версии и инструмента сборки Vite. Разработанное приложение представляет собой одностраничный сайт, отображающий персональный профиль пользователя и настраиваемый набор ссылок-карточек с уникальным анимированным оформлением в эстетике синтвейв.

Целью учебной практики является закрепление и практическое применение навыков разработки клиентских веб-приложений на основе компонентного подхода, а также формирование умений проектирования архитектуры программного продукта, работы с реактивными данными и создания выразительных анимированных пользовательских интерфейсов.

В соответствии с поставленной целью в ходе практики решались следующие задачи: анализ существующих решений в категории персональных витрин ссылок и выявление их недостатков; выбор и обоснование технологического стека; проектирование компонентной архитектуры приложения и структуры данных; реализация визуального интерфейса с многоуровневыми анимационными эффектами; обеспечение возможности простой кастомизации содержимого приложения без изменения его программного кода; тестирование всех реализованных функций и финальная сборка проекта.

Ожидаемым результатом практики являлось создание функционального, визуально привлекательного и легко настраиваемого веб-приложения с чётко структурированной кодовой базой, организованной по компонентному принципу и допускающей дальнейшее расширение функциональности. Приложение должно было быть готово к немедленному развёртыванию на статическом хостинге без необходимости настройки серверной части.

1 ПОСТАНОВКА ЗАДАЧИ (ТЕХНИЧЕСКОЕ ЗАДАНИЕ)

1.1 Цель проекта

Целью проекта является проектирование и разработка клиентского веб-приложения «Link Showcase», представляющего собой персональную витрину ссылок с поддержкой визуальной кастомизации. Приложение предназначено для агрегации ссылок на социальные сети, профессиональное портфолио, мессенджеры и контактные данные пользователя в рамках единой персональной страницы с уникальным дизайнерским оформлением.

Разрабатываемое приложение должно решать следующие практические задачи. Во-первых, предоставлять пользователю единую точку входа для всех его интернет-ресурсов, размещаемую в описании профиля социальной сети или в подписи электронного письма. Во-вторых, формировать у посетителя страницы положительное первое впечатление о её владельце посредством выразительного и запоминающегося визуального оформления. В-третьих, обеспечивать максимальную простоту настройки содержимого страницы, не требующую от пользователя знания языков программирования сверх минимального уровня, достаточного для редактирования конфигурационного файла.

Приложение должно отличаться от типовых шаблонных решений уникальным визуальным стилем, выбранным в эстетике синтвейв — направления ретрофутуристического дизайна, характеризующегося тёмным фоном, неоновыми акцентными цветами, перспективными сетками и анимированными эффектами свечения. Данный стиль обеспечивает высокую визуальную различимость страницы на фоне стандартных решений и делает её запоминаемой для посетителей.

Дополнительной целью проекта с точки зрения учебного процесса является практическое освоение компонентного подхода к разработке пользовательских интерфейсов, реализованного средствами Vue 3, а также приобретение навыков работы с современным инструментарием фронтенд-разработки, включая систему сборки Vite, однофайловые компоненты Vue и Composition API.

1.2 Анализ существующих аналогов

Перед началом проектирования приложения был проведён анализ наиболее распространённых сервисов класса «link-in-bio», представленных на рынке. Целью анализа являлось выявление типичных функциональных возможностей подобных сервисов, определение их недостатков и формулирование требований к разрабатываемому приложению с учётом выявленных ограничений.

В качестве объектов анализа были рассмотрены три наиболее популярных представителя данной категории: Linktree, Taplink и Bento. Linktree является пионером и наиболее известным представителем рынка персональных витрин ссылок. Сервис предоставляет простой интерфейс для создания страницы с набором кнопок-ссылок, поддерживает аналитику переходов и интеграцию с рядом платформ. Taplink ориентирован преимущественно на русскоязычную аудиторию и предоставляет

расширенные возможности для создания мини-сайтов с формами, мессенджерами и платёжными виджетами. Bento предлагает блочный подход к компоновке страницы, позволяя размещать различные типы виджетов в сеточном макете.

В таблице 1 представлено сравнение рассмотренных сервисов по ключевым характеристикам.

Таблица 1 – Сравнительный анализ существующих аналогов

№	**Характеристика**	**Linktree**	**Taplink**	**Bento**	**Link Showcase**
1	*2*	*3*	*4*	*5*	*6*
1	Бесплатный доступ	Частично	Частично	Частично	Полностью
2	Кастомизация дизайна	Ограничена	Ограничена	Ограничена	Полная
3	Уникальный визуальный стиль	Нет	Нет	Нет	Да
4	Зависимость от стороннего сервиса	Да	Да	Да	Нет
5	Хранение данных на своём сервере	Нет	Нет	Нет	Да
6	Открытый исходный код	Нет	Нет	Нет	Да
7	Необходимость регистрации	Да	Да	Да	Нет
Примечание — составлено автором на основании проведённого исследования					

По результатам анализа были выявлены следующие общие недостатки рассмотренных сервисов. Первым и наиболее существенным недостатком является ограниченность бесплатного тарифного плана. Во всех рассмотренных сервисах расширенные возможности кастомизации — выбор произвольных шрифтов, цветов, анимаций, скрытие брендинга сервиса — доступны исключительно в рамках платной подписки. Пользователь бесплатного тарифа вынужден довольствоваться ограниченным набором предустановленных тем оформления.

Вторым недостатком является визуальная однородность страниц, созданных на основе предустановленных шаблонов. Поскольку большинство пользователей сервисов используют одни и те же темы оформления, персональные страницы выглядят похоже и не создают у посетителя устойчивого визуального впечатления о личности владельца.

Третьим недостатком является полная зависимость пользователя от конкретного сервиса. При прекращении работы сервиса, изменении ценовой политики или блокировке аккаунта пользователь теряет доступ к своей странице. Данные профиля и настройки

хранятся на серверах сервиса и не могут быть перенесены на другую платформу без ручного воссоздания.

Четвёртым недостатком является обязательная регистрация и создание аккаунта, что предполагает передачу персональных данных стороннему сервису и согласие с его политикой конфиденциальности.

Разрабатываемое приложение «Link Showcase» устраняет все перечисленные недостатки. Приложение распространяется в виде исходного кода, который пользователь размещает на собственном хостинге. Это обеспечивает полный контроль над данными, отсутствие зависимости от сторонних сервисов и неограниченные возможности кастомизации. Уникальный визуальный стиль, реализованный в рамках проекта, обеспечивает высокую визуальную различимость страницы. Отсутствие серверной части и регистрации исключает необходимость передачи персональных данных.

1.3 Функциональные требования

На основании проведённого анализа существующих аналогов и сформулированной цели проекта авторами были определены функциональные требования к разрабатываемому приложению. Функциональные требования описывают конкретные возможности и поведение системы, которые должны быть реализованы в готовом продукте.

Требование 1. Отображение персонального профиля пользователя. Приложение должно отображать блок персональных данных владельца страницы, включающий аватар пользователя в виде изображения, загружаемого по URL-адресу, имя пользователя, псевдоним в формате дескриптора с символом «@» и краткое биографическое описание. В случае отсутствия изображения аватара приложение должно автоматически формировать и отображать инициалы пользователя, извлечённые из значения поля имени.

Требование 2. Отображение списка ссылок в виде карточек. Приложение должно отображать настраиваемый набор ссылок в виде интерактивных карточек, расположенных вертикальным списком. Количество отображаемых карточек не ограничивается и определяется исключительно содержимым конфигурационного файла. Каждая карточка должна содержать иконку, текстовую подпись и целевой URL-адрес. При нажатии на карточку целевой адрес должен открываться в новой вкладке браузера.

Требование 3. Поддержка цветовой дифференциации карточек. Для каждой карточки ссылки должна быть предусмотрена возможность независимого выбора цветового акцента оформления из заранее определённого набора значений. Цветовой акцент должен проявляться в цвете границы карточки, свечении при наведении, цвете иконки и цвете текста при наведении курсора.

Требование 4. Поддержка двух форматов иконок. Иконка карточки ссылки должна поддерживать два способа задания: в виде встроенной SVG-разметки, обеспечивающей масштабируемость и управление цветом, и в виде символа эмодзи, обеспечивающего простоту добавления без необходимости поиска SVG-кода.

Требование 5. Анимированный фоновый эффект. Приложение должно содержать непрерывно воспроизводящийся анимированный фоновый эффект, включающий

несколько визуальных слоёв. Анимации фона не должны блокировать взаимодействие пользователя с элементами интерфейса. При размонтировании компонентов все активные анимации и обработчики событий должны корректно завершаться.

Требование 6. Интерактивные эффекты карточек при наведении. При наведении курсора на карточку ссылки должен активироваться комплекс визуальных эффектов, включающий смещение и масштабирование карточки, появление неонового свечения границы, световой блик и кратковременный глитч-эффект текста. Эффекты должны плавно появляться и исчезать посредством CSS-переходов.

Требование 7. Каскадная анимация появления карточек. При загрузке страницы карточки списка ссылок должны появляться последовательно с нарастающей задержкой, создавая каскадный эффект. Задержка появления каждой последующей карточки должна быть больше задержки предыдущей на фиксированное значение.

Требование 8. Настройка содержимого через конфигурационный файл. Все данные профиля пользователя и параметры ссылок должны быть вынесены в единый конфигурационный файл. Редактирование данного файла должно полностью определять содержимое страницы без необходимости внесения изменений в какой-либо компонент приложения.

Требование 9. Адаптивность интерфейса. Приложение должно корректно отображаться на устройствах с различной шириной экрана, включая мобильные телефоны, планшеты и настольные компьютеры. Все элементы интерфейса должны сохранять читаемость и функциональность при любой ширине области просмотра.

В таблице 2 представлен перечень функциональных требований с указанием компонентов, ответственных за их реализацию.

Таблица 2 – Функциональные требования и реализующие компоненты

№	**Описание**	**Реализующий компонент**
1	*2*	*3*
1	Отображение профиля пользователя	ProfileHeader.vue
2	Список ссылок в виде карточек	LinkList.vue, LinkCard.vue
3	Цветовая дифференциация карточек	LinkCard.vue
4	Поддержка двух форматов иконок	LinkCard.vue
5	Анимированный фоновый эффект	BackgroundEffect.vue
6	Интерактивные эффекты при наведении	LinkCard.vue
7	Каскадная анимация появления	LinkList.vue, LinkCard.vue
8	Настройка через конфигурационный файл	links.js, App.vue
9	Адаптивность интерфейса	App.vue, все компоненты
Примечание — составлено автором на основании проведённого исследования		

1.4 Нефункциональные требования

Нефункциональные требования определяют качественные характеристики системы, не связанные напрямую с конкретными функциями, но оказывающие существенное влияние на удобство использования, сопровождения и дальнейшего развития приложения. В ходе проектирования авторами были сформулированы следующие нефункциональные требования к разрабатываемому приложению.

Требование к производительности. Приложение должно загружаться и становиться интерактивным в браузере пользователя за минимально возможное время. Поскольку приложение является полностью клиентским и не выполняет обращений к серверному API, основным фактором, влияющим на скорость загрузки, является размер итогового JavaScript-бандла. Данное требование обеспечивается за счёт использования инструмента сборки Vite, применяющего механизм tree-shaking — исключения неиспользуемого кода из итогового бандла — а также минификацию и сжатие всех файлов. Анимации приложения должны воспроизводиться с частотой не менее 60 кадров в секунду на устройствах со средними характеристиками. Для достижения данного показателя анимации реализуются преимущественно средствами CSS с использованием свойств transform и opacity, аппаратное ускорение которых поддерживается всеми современными браузерами.

Требование к поддерживаемости. Кодовая база проекта должна быть организована таким образом, чтобы любой разработчик с базовым знанием Vue 3 мог разобраться в структуре проекта и внести необходимые изменения за минимальное время. Данное требование обеспечивается следующими мерами: строгим соблюдением принципа единственной ответственности при проектировании компонентов — каждый компонент отвечает за единственную, чётко определённую область отображения; применением атрибута scoped в блоках стилей компонентов, исключающего нежелательное взаимодействие стилей между компонентами; использованием информативных имён переменных, параметров и классов; выносом всех данных в единый конфигурационный файл, являющийся единственным местом, требующим редактирования при изменении содержимого страницы.

Требование к масштабируемости. Архитектура приложения должна допускать добавление новых компонентов, функций и типов содержимого без необходимости переработки существующего кода. В частности, добавление новой ссылки должно сводиться исключительно к добавлению нового объекта в массив links конфигурационного файла. Добавление нового цветового акцента карточки должно требовать лишь добавления нового CSS-класса-модификатора в компонент LinkCard.vue без изменения логики компонента. Расширение полей профиля пользователя должно требовать добавления соответствующего поля в объект profile конфигурационного файла и нового элемента разметки в компонент ProfileHeader.vue.

Требование к доступности. Интерактивные элементы приложения должны быть реализованы с использованием семантически корректных HTML-элементов. В частности, карточки ссылок должны быть реализованы посредством элемента якорной ссылки с корректными атрибутами, а не посредством элементов div с обработчиками событий.

Декоративные элементы интерфейса, не несущие смысловой нагрузки, должны быть скрыты от вспомогательных технологий посредством атрибута aria-hidden. Изображение аватара должно содержать атрибут alt с именем пользователя.

Требование к кроссбраузерности. Приложение должно корректно функционировать и отображаться в актуальных версиях браузеров Google Chrome, Mozilla Firefox и Microsoft Edge. Использование экспериментальных CSS-свойств, не поддерживаемых данными браузерами, не допускается. В случае применения свойств, требующих вендорных префиксов, необходимые префиксы должны быть добавлены явно в коде стилей.

Требование к независимости от серверной части. Приложение не должно требовать наличия серверной части для своей работы. Все необходимые данные должны содержаться в исходном коде приложения. Итоговый бандл после сборки должен представлять собой набор статических файлов, размещение которых возможно на любом хостинге статических сайтов, включая бесплатные платформы такие как GitHub Pages и Netlify.

В таблице 3 представлен перечень нефункциональных требований с указанием способа их обеспечения.

Таблица 3 – Нефункциональные требования и способы их обеспечения

№	**Требование**	**Способ обеспечения**
1	*2*	*3*
1	Производительность	Vite, tree-shaking, CSS-анимации на transform и opacity
2	Поддерживаемость	Принцип единственной ответственности, scoped-стили, конфигурационный файл
3	Масштабируемость	Компонентная архитектура, CSS-классы-модификаторы
4	Доступность	Семантические HTML-элементы, атрибуты aria-hidden и alt
5	Кроссбраузерность	Вендорные префиксы, отказ от экспериментальных свойств
6	Независимость от сервера	Статическая сборка, отсутствие серверного API
Примечание — составлено автором на основании проведённого исследования		

1.5 Критерии готовности

Критерии готовности представляют собой чётко определённый перечень условий, при выполнении которых проект считается завершённым и готовым к защите. Формулирование критериев готовности до начала разработки позволяет избежать

неопределённости при оценке результатов и обеспечивает объективную основу для проверки полноты реализации всех требований.

Критерий 1. Реализация всех функциональных требований. Все девять функциональных требований, сформулированных в разделе 1.3, должны быть реализованы и проверены в браузере. Проверка каждого требования должна подтверждаться визуальным наблюдением ожидаемого поведения приложения в ходе тестирования.

Критерий 2. Соответствие компонентной структуры спроектированной архитектуре. Приложение должно состоять из пяти файлов компонентов и одного конфигурационного файла данных в соответствии со структурой, определённой на этапе проектирования. Каждый компонент должен выполнять исключительно ту функцию, которая определена для него архитектурой приложения, не содержа логики, относящейся к ответственности других компонентов.

Критерий 3. Корректность передачи данных через конфигурационный файл. Изменение любого поля в конфигурационном файле links.js должно немедленно отражаться в интерфейсе приложения без необходимости внесения изменений в какой-либо компонент. Добавление нового объекта в массив links должно приводить к появлению соответствующей карточки в списке ссылок. Удаление объекта из массива должно приводить к исчезновению соответствующей карточки.

Критерий 4. Работоспособность всех анимационных эффектов. Все реализованные анимации — вращение кольца аватара, движение перспективной сетки, плавание частиц, каскадное появление карточек, световой блик и глитч-эффект при наведении — должны воспроизводиться корректно без визуальных сбоев и артефактов. Анимации не должны влиять на доступность интерактивных элементов страницы.

Критерий 5. Корректная работа при отсутствии необязательных параметров. Приложение должно корректно функционировать при пустом значении поля avatar объекта profile, отображая инициалы пользователя вместо изображения. Приложение должно корректно функционировать при отсутствии значения поля color в объекте ссылки, применяя акцент по умолчанию.

Критерий 6. Адаптивность на всех целевых разрешениях. Интерфейс приложения должен корректно отображаться и оставаться полностью функциональным при ширине области просмотра от 320 пикселей, соответствующей минимальной ширине мобильных устройств, до 1920 пикселей, соответствующей стандартному разрешению широкоформатных мониторов. Все текстовые элементы должны оставаться читаемыми при любой ширине без горизонтальной прокрутки.

Критерий 7. Успешная сборка проекта. Выполнение команды сборки npm run build должно завершаться без ошибок и предупреждений. Сформированный бандл в директории dist должен содержать корректные минифицированные файлы JavaScript и CSS. Открытие файла index.html из директории dist в браузере должно приводить к корректному отображению приложения.

Критерий 8. Корректное завершение анимаций при изменении состояния. При изменении размера окна браузера холст анимации частиц должен корректно пересчитывать свои размеры. При программном удалении компонента фонового эффекта из дерева компонентов цикл анимации частиц должен корректно останавливаться, не

оставляя активных обработчиков событий и незавершённых циклов requestAnimationFrame.

В таблице 4 представлен перечень критериев готовности с указанием метода проверки каждого из них.

Таблица 4 – Критерии готовности и методы их проверки

№	**Критерий**	**Метод проверки**
1	*2*	*3*
1	Реализация функциональных требований	Визуальная проверка в браузере
2	Соответствие компонентной структуры	Сверка с проектной документацией
3	Корректность передачи данных	Редактирование конфигурационного файла и наблюдение результата
4	Работоспособность анимаций	Визуальная проверка в браузере
5	Поведение при отсутствии параметров	Очистка необязательных полей и наблюдение результата
6	Адаптивность	Проверка в инструментах разработчика браузера при различных разрешениях
7	Успешная сборка	Выполнение команды npm run build и проверка директории dist
8	Корректное завершение анимаций	Изменение размера окна и наблюдение поведения холста
Примечание — составлено автором на основании проведённого исследования		

2 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

2.1 Выбор технологического стека и его обоснование

Технологический стек проекта — совокупность инструментов, фреймворков и библиотек, применяемых при разработке приложения, — определяет не только возможности конечного продукта, но и удобство процесса разработки, скорость итераций и долгосрочную поддерживаемость кодовой базы. Выбор технологического стека производился на основании анализа требований проекта, актуальности технологий в индустрии и соответствия уровню подготовки авторов.

В качестве основного фреймворка для построения пользовательского интерфейса был выбран Vue 3. Vue является прогрессивным JavaScript-фреймворком, что означает возможность его постепенного внедрения в проект — от простого добавления реактивности к отдельным элементам страницы до построения полноценного одностраничного приложения с маршрутизацией и управлением состоянием. Для данного проекта Vue 3 применяется в своей наиболее полной форме: как основа компонентной архитектуры одностраничного приложения.

Принципиальным отличием Vue 3 от предшествующей версии является наличие Composition API — альтернативного способа организации логики компонентов. В отличие от Options API, организующего код по категориям — данные, вычисляемые свойства, методы, хуки жизненного цикла, — Composition API позволяет группировать связанный код вместе независимо от его категории. Это особенно ценно в компонентах со сложной логикой, таких как BackgroundEffect.vue, где инициализация холста, цикл анимации, обработчик изменения размера окна и логика очистки при размонтировании компонента тесно связаны между собой и должны быть сосредоточены в одном месте для удобства чтения и сопровождения.

Важной особенностью Vue 3, использованной в проекте, является синтаксис script setup — сокращённая форма записи Composition API, при которой все объявления верхнего уровня блока script автоматически становятся доступными в шаблоне компонента без явного возврата через объект. Данный синтаксис сокращает объём шаблонного кода и делает компоненты более лаконичными.

В качестве инструмента сборки был выбран Vite. Данный инструмент разработан Эваном Ю — создателем Vue — и является официально рекомендованным способом создания новых Vue-проектов начиная с выхода Vue 3. Ключевым преимуществом Vite в режиме разработки является использование нативных ES-модулей браузера вместо предварительной сборки всего кода приложения в единый бандл. Это обеспечивает практически мгновенный запуск сервера разработки вне зависимости от размера проекта и значительно ускоряет применение изменений при редактировании файлов посредством механизма горячей замены модулей. В режиме производственной сборки Vite использует Rollup для формирования оптимизированного бандла с разделением кода, минификацией и удалением неиспользуемого кода.

Стилизация компонентов приложения реализована средствами нативного CSS без применения дополнительных библиотек или препроцессоров. Данное решение обосновано тем, что современный CSS обладает всеми необходимыми для реализации

требований проекта возможностями: переменными для хранения значений палитры, анимациями и переходами для интерактивных эффектов, трансформациями для создания перспективной сетки и эффектов наведения, функцией clamp для адаптивной типографики, свойствами backdrop-filter и filter для создания эффектов свечения и размытия. Применение нативного CSS исключает необходимость подключения дополнительных зависимостей и уменьшает итоговый размер бандла.

В таблице 5 представлен итоговый технологический стек проекта с обоснованием выбора каждого инструмента.

Таблица 5 – Технологический стек проекта

№	**Инструмент**	**Версия**	**Назначение**	**Обоснование выбора**
1	*2*	*3*	*4*	*5*
1	Vue	3	Фреймворк пользовательского интерфейса	Актуальная версия, Composition API, однофайловые компоненты
2	Vite	Актуальная	Инструмент сборки	Официально рекомендован для Vue 3, высокая скорость
3	Node.js	20	Среда выполнения для инструментов разработки	Требуется для работы Vite и npm
4	npm	Актуальный	Менеджер пакетов	Поставляется в комплекте с Node.js
5	CSS	Нативный	Стилизация компонентов	Достаточность возможностей, отсутствие лишних зависимостей
6	Google Fonts	—	Подключение шрифтов	Бесплатный доступ, широкий выбор, CDN-доставка
7	Canvas API	—	Анимация частиц	Нативный браузерный API, не требует зависимостей
Примечание — составлено автором на основании проведённого исследования				

2.2 Проектирование структуры данных

Проектирование структуры данных приложения предшествовало написанию любого программного кода и являлось одним из ключевых этапов подготовительной работы.

Несмотря на то что разрабатываемое приложение является полностью клиентским и не использует серверную базу данных, задача проектирования структуры данных не теряет своей актуальности: необходимо было определить состав, типы и организацию всех данных, которыми оперирует приложение, а также способ их хранения и передачи между компонентами.

В ходе анализа предметной области были выделены два логически самостоятельных объекта данных. Первый объект описывает персональные данные владельца страницы и был назван profile. Второй объект представляет собой коллекцию ссылок, отображаемых на странице, и был реализован в виде массива links, каждый элемент которого описывает одну карточку-ссылку.

Выбор в пользу хранения данных в виде статического JavaScript-модуля, а не в формате JSON-файла или внешнего API, обусловлен несколькими соображениями. Во-первых, JavaScript-модуль допускает использование многострочных шаблонных строк для хранения SVG-иконок, тогда как формат JSON не поддерживает многострочные строки и требует экранирования специальных символов. Во-вторых, JavaScript-модуль может содержать комментарии, поясняющие назначение полей и допустимые значения параметров, что существенно упрощает редактирование конфигурации пользователями без глубоких технических знаний. В-третьих, импорт JavaScript-модуля обрабатывается сборщиком Vite статически, что позволяет включить данные непосредственно в итоговый бандл без дополнительных сетевых запросов при загрузке страницы.

Объект profile содержит четыре поля. Поле avatar хранит строковое значение URL-адреса изображения аватара пользователя. Данное поле является необязательным: при пустом значении компонент ProfileHeader автоматически переключается в режим отображения инициалов. Поле name хранит строковое значение отображаемого имени пользователя и является обязательным. Поле handle хранит строковое значение псевдонима пользователя, отображаемого с префиксом «@», и является обязательным. Поле bio хранит строковое значение биографического описания и является необязательным.

Каждый элемент массива links описывается пятью полями. Поле id хранит строковый уникальный идентификатор записи, используемый Vue в качестве значения атрибута key при рендеринге списка для эффективного обновления DOM. Поле label хранит строковое значение текста, отображаемого на карточке ссылки. Поле url хранит целевой адрес ссылки в виде строки и поддерживает как стандартные адреса с протоколом https, так и адреса формата mailto для ссылок на электронную почту. Поле icon хранит строковое значение иконки в одном из двух допустимых форматов. Поле color хранит строковое значение цветового акцента карточки и принимает одно из четырёх допустимых значений: pink, cyan, purple или yellow.

В таблице 6 представлено детальное описание полей объекта profile.

Таблица 6 – Описание полей объекта profile

№	Поле	Тип данных	Обязательность	Значение по умолчанию	Описание
1	*2*	*3*	*4*	*5*	*6*
1	avatar	String	Нет	Пустая строка	URL-адрес изображения аватара пользователя
2	name	String	Да	Отсутствует	Отображаемое имя пользователя
3	handle	String	Да	Отсутствует	Псевдоним пользователя без символа «@»
4	bio	String	Нет	Пустая строка	Краткое биографическое описание
Примечание — составлено автором на основании проведённого исследования					

В таблице 7 представлено детальное описание полей элемента массива links.

Таблица 7 – Описание полей элемента массива links

№	Поле	Тип данных	Обязательность	Допустимые значения	Описание
1	*2*	*3*	*4*	*5*	*6*
1	id	String	Да	Любая уникальная строка	Уникальный идентификатор записи
2	label	String	Да	Любая строка	Текст карточки ссылки
3	url	String	Да	https-адрес или mailto-ссылка	Целевой адрес при нажатии
4	icon	String	Да	SVG-строка или символ эмодзи	Иконка карточки
5	color	String	Нет	pink, cyan, purple, yellow	Цветовой акцент карточки

Продолжение таблицы 7

Примечание — составлено автором на основании проведённого исследования

Для обеспечения консистентности данных при редактировании конфигурационного файла в его коде предусмотрены комментарии, поясняющие допустимые значения каждого поля. Это позволяет пользователю, не знакомому с исходным кодом компонентов, корректно настраивать содержимое страницы, опираясь исключительно на подсказки в конфигурационном файле.

Отдельного внимания заслуживает решение о поддержке двух форматов иконок в рамках одного поля icon. Первый формат — встроенная SVG-разметка — обеспечивает наилучшее качество отображения, масштабируемость и возможность управления цветом иконки через CSS-свойство color, что позволяет автоматически применять к иконке акцентный цвет соответствующей карточки. Второй формат — символ эмодзи — не поддерживает управление цветом, однако существенно упрощает добавление новых ссылок: пользователю достаточно вставить символ эмодзи непосредственно в строковое значение поля без необходимости поиска и копирования SVG-кода. Компонент LinkCard отображает значение поля icon посредством директивы v-html, корректно обрабатывающей оба формата.

2.3 Проектирование компонентной архитектуры

Компонентная архитектура является фундаментальным принципом построения приложений на Vue и большинстве других современных фреймворков для разработки пользовательских интерфейсов. Суть данного принципа состоит в разделении интерфейса на независимые, повторно используемые части — компоненты, каждый из которых инкапсулирует собственную разметку, логику и стили. Проектирование компонентной архитектуры предшествует написанию кода и определяет, на какие части будет разбит интерфейс, как компоненты будут взаимодействовать между собой и каким образом данные будут передаваться по дереву компонентов.

При проектировании архитектуры приложения «Link Showcase» авторами применялся подход декомпозиции интерфейса сверху вниз. На первом уровне декомпозиции был выделен корневой компонент App.vue, являющийся точкой входа в дерево компонентов и отвечающий за общую компоновку страницы и передачу данных дочерним компонентам. На втором уровне были выделены четыре дочерних компонента, каждый из которых отвечает за самостоятельную визуальную зону страницы. На третьем уровне был выделен компонент отдельной карточки ссылки, являющийся дочерним по отношению к компоненту списка ссылок.

Руководящим принципом при определении границ компонентов служил принцип единственной ответственности: каждый компонент должен иметь одну чётко определённую причину для изменения. Применение данного принципа привело к следующему разбиению интерфейса на компоненты.

Компонент BackgroundEffect.vue выделен в отдельную единицу по той причине, что он является полностью самодостаточным визуальным эффектом, не зависящим от каких-либо внешних данных и не влияющим на отображение других элементов страницы. Вся логика анимации — инициализация холста, цикл отрисовки частиц, обработка изменения размера окна — сосредоточена внутри компонента и не просачивается наружу. Это означает, что данный компонент может быть безопасно заменён альтернативной реализацией фонового эффекта без каких-либо изменений в других компонентах приложения.

Компонент ProfileHeader.vue выделен в отдельную единицу по той причине, что он представляет собой логически завершённый блок отображения персональных данных пользователя. Все элементы блока — аватар, имя, псевдоним и биографическое описание — тесно связаны между собой тематически и визуально, образуя единое целое. Параметры компонента представляют собой прямое отображение полей объекта profile конфигурационного файла.

Компонент LinkList.vue выделен в отдельную единицу, отвечающую исключительно за рендеринг списка карточек и управление задержками их анимации появления. Вынесение логики списка в отдельный компонент освобождает корневой компонент App.vue от необходимости знать о том, как именно организован рендеринг коллекции ссылок, и позволяет в будущем изменить компоновку карточек — например, перейти от вертикального списка к сеточному макету — путём модификации исключительно данного компонента.

Компонент LinkCard.vue выделен в отдельную единицу по той причине, что он инкапсулирует всю сложную логику интерактивного поведения отдельной карточки: систему цветовых акцентов, эффекты наведения, световой блик, глитч-эффект и анимацию появления. Изоляция данной логики в отдельном компоненте обеспечивает её переиспользуемость: один и тот же компонент используется для отображения всех карточек списка вне зависимости от их количества и содержимого.

В таблице 8 представлено описание всех компонентов приложения с указанием их ответственности, входных параметров и дочерних компонентов.

Таблица 8 – Описание компонентов приложения

№	**Компонент**	**Ответственность**	**Входные параметры**	**Дочерние компоненты**
1	*2*	*3*	*4*	*5*
1	App.vue	Компоновка страницы, передача данных	Отсутствуют	kgroundEffect, ProfileHeader, LinkList
2	BackgroundEffect.vue	Анимированный фоновый эффект	Отсутствуют	Отсутствуют
3	ProfileHeader.vue	Отображение профиля пользователя	avatar, name, handle, bio	Отсутствуют

Продолжение таблицы 8

4	LinkList.vue	Рендеринг списка карточек	links	LinkCard
5	LinkCard.vue	Отображение одной карточки-ссылки	link, delay	Отсутствуют
Примечание — составлено автором на основании проведённого исследования				

Взаимодействие компонентов строится исключительно на механизме передачи данных сверху вниз через входные параметры — props. Данный паттерн, называемый однонаправленным потоком данных, является рекомендованным подходом в Vue и обеспечивает предсказуемость поведения приложения: любое изменение данных всегда инициируется на верхнем уровне дерева компонентов и распространяется вниз, что исключает неочевидные побочные эффекты. Поскольку приложение не предполагает пользовательского ввода, изменяющего состояние данных, необходимость в механизмах передачи событий снизу вверх и в централизованном управлении состоянием отсутствует.

Схема передачи данных в приложении выглядит следующим образом. Конфигурационный файл links.js импортируется корневым компонентом App.vue. Корневой компонент передаёт поля объекта profile в компонент ProfileHeader.vue в виде отдельных именованных параметров. Массив links передаётся в компонент LinkList.vue единым параметром. Компонент LinkList.vue в цикле рендеринга передаёт каждый элемент массива в компонент LinkCard.vue вместе с вычисленным значением задержки анимации. Компонент BackgroundEffect.vue не получает никаких данных извне и функционирует полностью автономно.

2.4 Внешняя структура приложения

Приложение «Link Showcase» реализовано по архитектуре одностраничного приложения и не предполагает многостраничной навигации. Вся функциональность сосредоточена на единственной странице, загружаемой при обращении к корневому маршруту. Взаимодействие пользователя с приложением сводится к единственному типу действия — нажатию на карточку ссылки, открывающему целевой адрес в новой вкладке браузера.

Визуальное пространство страницы разделено на четыре зоны, расположенные в следующем порядке сверху вниз: зона фонового эффекта, зона профиля пользователя, зона списка ссылок и зона нижнего колонтитула. Зона фонового эффекта занимает всё пространство области просмотра и располагается позади всех остальных элементов страницы, не препятствуя взаимодействию с ними.

В таблице 9 представлено описание визуальных зон страницы с указанием их назначения и доступных действий.

Таблица 9 – Внешняя структура приложения

№	Зона	Назначение	Доступные действия пользователя
1	*2*	*3*	*4*
1	Фоновый эффект	Визуальная атмосфера страницы	Отсутствуют
2	Профиль пользователя	Отображение аватара, имени, псевдонима и биографии	Отсутствуют
3	Список ссылок	Отображение карточек-ссылок	Переход по ссылке при нажатии на карточку
4	Нижний колонтитул	Декоративная подпись страницы	Отсутствуют
Примечание — составлено автором на основании проведённого исследования			

2.5 Внутренняя структура приложения

Внутренняя структура приложения организована в соответствии с принятыми в экосистеме Vue 3 соглашениями об именовании и расположении файлов. Корневая директория проекта содержит конфигурационные файлы инструментов разработки, а вся прикладная логика сосредоточена в директории src.

Файл index.html является точкой входа всего приложения. Он содержит минимальную HTML-разметку: объявление типа документа, мета-теги с настройками кодировки и области просмотра, заголовок страницы и корневой элемент div с идентификатором app. В данный элемент Vue монтирует всё дерево компонентов при инициализации приложения. Кроме того, в файле размещён тег script, подключающий файл инициализации main.js как ES-модуль.

Файл src/main.js выполняет инициализацию приложения. В нём производится импорт функции createApp из пакета vue, импорт корневого компонента App.vue, создание экземпляра приложения и его монтирование в DOM-элемент с идентификатором app. Данный файл намеренно содержит минимум кода: поскольку приложение не использует маршрутизацию и централизованное управление состоянием, необходимость в регистрации плагинов отсутствует.

Файл src/App.vue является корневым компонентом приложения. В блоке script setup производится импорт всех четырёх дочерних компонентов и деструктурированный импорт объектов profile и links из конфигурационного файла. В блоке template определена общая компоновка страницы. В блоке style без атрибута scoped размещены глобальные стили,

доступные всем компонентам приложения: сброс отступов, объявление CSS-переменных, базовые свойства документа и стили контейнера.

Файл src/data/links.js является единственным файлом, который пользователь редактирует для настройки содержимого страницы. Он экспортирует объект profile с данными профиля и массив links с описанием карточек ссылок. Файл содержит подробные комментарии, поясняющие назначение каждого поля и допустимые значения параметров.

Директория src/components содержит четыре файла однофайловых компонентов. Файл BackgroundEffect.vue реализует анимированный фоновый эффект из четырёх слоёв. Файл ProfileHeader.vue реализует блок отображения персональных данных пользователя. Файл LinkList.vue реализует контейнер для рендеринга списка карточек. Файл LinkCard.vue реализует отдельную интерактивную карточку-ссылку со всеми эффектами наведения.

Файл vite.config.js содержит конфигурацию инструмента сборки. В нём подключается плагин Vue для Vite, обеспечивающий корректную обработку однофайловых компонентов с расширением .vue. Все остальные настройки сборки оставлены в значениях по умолчанию, которые являются оптимальными для проектов данного типа и размера.

После выполнения команды сборки формируется директория dist, содержащая готовые к развёртыванию файлы: index.html, минифицированный JavaScript-бандл и минифицированный CSS-файл. Данная директория представляет собой полностью самодостаточный набор статических файлов, не требующий для своей работы ни серверной части, ни установленного Node.js на хостинге.

3 ПРАКТИЧЕСКАЯ ЧАСТЬ РЕАЛИЗАЦИИ ПРОЕКТА

3.1 Настройка проекта и рабочего окружения

Разработка приложения началась с подготовки рабочего окружения и инициализации проекта. На рабочей машине была проверена наличие актуальной версии Node.js, являющейся обязательным условием для работы инструментов разработки. Node.js предоставляет среду выполнения JavaScript вне браузера и включает в себя менеджер пакетов npm, посредством которого устанавливаются все зависимости проекта и запускаются команды сборки.

Инициализация проекта выполнялась посредством официального инструмента создания Vue-приложений на базе Vite. После выполнения команды инициализации был выбран шаблон Vue с использованием JavaScript без TypeScript, поскольку добавление строгой типизации для проекта данного масштаба избыточно и увеличило бы время разработки без существенных преимуществ. В результате инициализации была сформирована стандартная структура директорий с файлами конфигурации, точкой входа и демонстрационными компонентами шаблона.

После инициализации из директории src были удалены все демонстрационные компоненты и стили, поставляемые шаблоном Vite: компонент HelloWorld.vue, стили base.css и main.css, а также демонстрационное содержимое корневого компонента App.vue. Данный шаг необходим для того, чтобы начать разработку с чистой кодовой базы, не содержащей лишних файлов и стилей, способных вступить в конфликт с оформлением разрабатываемого приложения.

Параллельно была создана директория src/components для размещения компонентов приложения и директория src/data для размещения конфигурационного файла данных. Создание отдельной директории для данных, несмотря на то что конфигурационный файл является единственным файлом в ней, соответствует принятым соглашениям о структуре Vue-проектов и обеспечивает логическое разделение между компонентами и данными.

Для подключения шрифтов в блоке глобальных стилей корневого компонента была добавлена директива импорта CSS из сервиса Google Fonts. Запрос к сервису включал два семейства шрифтов: Orbitron с тремя начертаниями для отображения заголовков и подписей карточек и Share Tech Mono с одним начертанием для основного текста. Выбор данных шрифтов обусловлен их соответствием эстетике синтвейв: Orbitron обладает геометрическими формами и футуристическим характером, Share Tech Mono имитирует текст терминала и усиливает технологическую атмосферу страницы.

Глобальные CSS-переменные были объявлены в псевдоклассе :root блока глобальных стилей. Переменные содержат полную цветовую палитру приложения: два оттенка тёмного фона, четыре акцентных неоновых цвета, параметры фона и границы карточек, а также два значения цвета текста. Централизованное объявление переменных обеспечивает единообразие оформления во всех компонентах и позволяет изменить цветовую схему всего приложения путём редактирования единственного блока кода.

3.2 Разработка конфигурационного файла данных

Разработка конфигурационного файла данных стала первым содержательным шагом после настройки окружения. Файл src/data/links.js был создан до написания кода каких-либо компонентов — это позволило чётко определить форму данных, которую компоненты будут получать в виде входных параметров, и избежать последующих переработок интерфейса компонентов.

Файл использует именованный экспорт двух объектов: profile и links. Выбор именованного экспорта вместо экспорта по умолчанию обусловлен тем, что он делает зависимость явной на стороне импортирующего модуля: в корневом компоненте App.vue сразу видно, что из файла извлекаются именно два конкретных объекта, а не некий безымянный объект по умолчанию. Это улучшает читаемость кода и упрощает навигацию по проекту.

Объект profile был заполнен тестовыми данными, отражающими типичный сценарий использования приложения. Поле avatar было оставлено пустой строкой намеренно, чтобы проверить корректность отображения инициалов в компоненте ProfileHeader на раннем этапе разработки. Поле bio содержит многострочное значение с переносом строки, что позволило убедиться в корректной обработке данного случая в компоненте отображения.

Массив links был заполнен шестью объектами, представляющими наиболее типичные для подобных страниц ссылки: профиль на GitHub, канал в Telegram, аккаунт в Twitter, канал на YouTube, личное портфолио и адрес электронной почты в формате mailto. Подобный набор ссылок охватывает оба поддерживаемых формата иконок — SVG-разметку для первых четырёх ссылок и символы эмодзи для двух последних — и все четыре допустимых значения цветового акцента, что позволило проверить корректность работы системы цветовых модификаторов в процессе разработки компонента LinkCard.

SVG-иконки для ссылок GitHub, Telegram, Twitter и YouTube были реализованы в виде встроенной разметки, размещённой непосредственно в строковых значениях поля icon. Каждая иконка представляет собой элемент svg с атрибутами viewBox, width и height, содержащий один или несколько элементов path с соответствующими данными контура. Атрибут fill каждой иконки установлен в значение currentColor, что обеспечивает наследование цвета из CSS-свойства color родительского элемента и позволяет компоненту LinkCard управлять цветом иконки посредством CSS-переменной акцентного цвета.

Файл содержит подробные однострочные комментарии над каждым полем объекта profile и над массивом links в целом, поясняющие назначение поля, тип ожидаемого значения и допустимые варианты. Данные комментарии адресованы конечному пользователю приложения и позволяют ему самостоятельно настроить содержимое страницы, не обращаясь к документации и не изучая код компонентов.

3.3 Разработка корневого компонента и глобальных стилей

Корневой компонент App.vue выполняет роль точки сборки всего приложения. Его задача состоит не в реализации какой-либо сложной логики, а в объединении дочерних компонентов в единую страницу и обеспечении потока данных от конфигурационного

файла к компонентам отображения. Намеренная простота корневого компонента является признаком правильно спроектированной архитектуры: вся содержательная логика делегирована специализированным дочерним компонентам.

В блоке script setup производится импорт четырёх дочерних компонентов и деструктурированный импорт объектов profile и links из конфигурационного файла. Поскольку данные конфигурационного файла являются статическими и не изменяются в процессе работы приложения, они не оборачиваются в реактивные обёртки ref или reactive. Это намеренное решение: использование реактивности для данных, которые никогда не изменятся, было бы избыточным и привело бы к незначительным, но бессмысленным накладным расходам.

В блоке template определена следующая структура: корневой элемент div с классом app содержит компонент BackgroundEffect без каких-либо атрибутов, элемент main с классом container, внутри которого последовательно размещены компоненты ProfileHeader и LinkList, а также элемент footer с декоративной подписью. Поля объекта profile передаются в компонент ProfileHeader в виде отдельных именованных атрибутов, а не в виде единого объекта. Данное решение обусловлено тем, что передача отдельных полей делает интерфейс компонента явным и самодокументируемым: из шаблона корневого компонента сразу видно, какие именно данные использует ProfileHeader.

Блок style корневого компонента не содержит атрибута scoped, что делает объявленные в нём стили глобальными. Это единственный компонент в проекте с глобальными стилями — все остальные компоненты используют изолированные стили с атрибутом scoped. Глобальный блок стилей содержит три группы объявлений. Первая группа — директива импорта шрифтов Google Fonts. Вторая группа — объявление CSS-переменных в псевдоклассе :root, содержащее полную цветовую палитру и значения шрифтовых семейств. Третья группа — базовые стили документа: сброс отступов и полей всех элементов посредством универсального селектора, установка цвета фона и шрифта для элементов html и body, стили корневого элемента app и контейнера main, а также стили нижнего колонтитула с анимацией сердечка.

Анимация сердечка в нижнем колонтитуле реализована CSS-анимацией heartbeat, применённой к элементу span, содержащему символ сердца. Анимация плавно увеличивает масштаб элемента до 1.3 в средней точке цикла и возвращает его к исходному размеру, создавая эффект пульсирующего сердцебиения с периодом 1.8 секунды.

3.4 Разработка компонента фонового эффекта

Компонент BackgroundEffect.vue является наиболее технически сложным в приложении. Он позиционируется фиксированно относительно области просмотра, занимает всё доступное пространство и располагается на нижнем уровне слоёв отображения посредством свойства z-index, не перекрывая интерактивные элементы страницы. Атрибут aria-hidden со значением true скрывает компонент от вспомогательных технологий, поскольку он является исключительно декоративным.

Первый визуальный слой — градиентные свечения — реализован тремя элементами div с абсолютным позиционированием и большим радиусом размытия посредством

свойства filter: blur. Каждый элемент использует радиальный градиент с одним из акцентных цветов палитры, угасающий к прозрачности. Два из трёх элементов анимированы CSS-анимациями driftL и driftR, плавно смещающими их по осям координат в режиме alternate, что создаёт эффект медленно дрейфующих источников неонового света.

Второй слой — сканирующие линии — реализован единственным элементом div, занимающим всё пространство компонента. Его фон задан повторяющимся линейным градиентом, чередующим прозрачные и полупрозрачные чёрные горизонтальные полосы шириной по два пикселя. Результирующая текстура имитирует растровую структуру экрана электронно-лучевой трубки, добавляя ретро-характер визуальному стилю страницы.

Третий слой — перспективная сетка — реализован двумя вложенными элементами div. Внешний элемент создаёт контекст перспективной проекции посредством свойства perspective. Внутренний элемент поворачивается вокруг горизонтальной оси посредством трансформации rotateX(60deg), создавая иллюзию горизонтальной плоскости, уходящей вдаль. Фон внутреннего элемента задан пересечением двух повторяющихся линейных градиентов, формирующих сеточный рисунок с акцентным фиолетовым цветом. CSS-анимация scrollGrid бесконечно смещает положение фона по оси Y, имитируя движение вперёд. Свойство mask-image обеспечивает плавное угасание сетки в направлении от нижнего края к верхнему, создавая естественное слияние сетки с тёмным фоном.

Четвёртый слой — анимированные частицы — реализован посредством элемента canvas и скрипта на JavaScript, написанного с применением Composition API. Ссылка на элемент canvas получается через реактивную переменную ref. Вся логика инициализируется в хуке жизненного цикла onMounted, вызываемом после монтирования компонента в DOM. При инициализации создаётся массив из 55 объектов частиц со случайными значениями координат, радиуса, вектора скорости, оттенка цвета и прозрачности. Координаты частиц хранятся в нормализованном виде — в диапазоне от нуля до единицы — что позволяет корректно масштабировать их положение при изменении размера окна без пересчёта координат каждой частицы. В каждом кадре анимации координаты частиц обновляются согласно вектору скорости, после чего частица отрисовывается в виде закрашенного круга цветом в модели HSL. При выходе частицы за пределы холста она появляется с противоположной стороны, обеспечивая непрерывность анимации. В хуке onUnmounted цикл анимации останавливается посредством cancelAnimationFrame и отвязывается обработчик события resize, что исключает утечки памяти при возможном размонтировании компонента.

3.5 Разработка компонента шапки профиля

Компонент ProfileHeader.vue отвечает за отображение персональных данных пользователя и принимает четыре входных параметра: avatar, name, handle и bio. Параметры объявляются посредством макроса defineProps с указанием типов и значений по умолчанию для необязательных полей.

Центральным элементом компонента является блок аватара. Вращающееся кольцо вокруг изображения реализовано средствами CSS без применения JavaScript. Внешний элемент кольца использует конический градиент conic-gradient, последовательно переходящий через розовый, фиолетовый и голубой акцентные цвета палитры. CSS-анимация spinRing выполняет полный оборот элемента за шесть секунд с линейной функцией временного хода и бесконечным повторением. Внутренний элемент имеет фон, совпадающий с основным цветом фона страницы, создавая визуальный эффект кольца с просветом вокруг изображения. Условный рендеринг блока аватара реализован посредством директив v-if и v-else: при непустом значении параметра avatar отображается элемент img с соответствующим атрибутом src и alt, при пустом — блок с инициалами. Инициалы вычисляются посредством computed-свойства: строка name разбивается по пробелам, из каждого слова извлекается первый символ, символы объединяются, переводятся в верхний регистр и обрезаются до двух символов.

Имя пользователя отображается с применением градиентного эффекта текста. Данный эффект реализован посредством трёх CSS-свойств в совокупности: свойство background задаёт горизонтальный линейный градиент от розового к голубому, свойство background-clip со значением text ограничивает отображение фона контуром символов, свойство -webkit-text-fill-color со значением transparent делает стандартную заливку символов прозрачной, открывая градиентный фон. Дополнительное свечение имени обеспечивается свойством filter: drop-shadow с розовым цветом и небольшим радиусом размытия. Размер шрифта имени задан посредством функции clamp с тремя значениями: минимальным, предпочтительным в единицах vw и максимальным, что обеспечивает плавное масштабирование заголовка на экранах различной ширины без медиа-запросов.

Псевдоним пользователя отображается шрифтом меньшего размера с акцентным фиолетовым цветом и увеличенным межбуквенным интервалом. Биографическое описание выводится приглушённым цветом с увеличенной высотой строки, обеспечивающей комфортное чтение многострочного текста. Декоративный разделитель в нижней части компонента состоит из двух линий с градиентной прозрачностью и центрального символа ромба, визуально отделяя блок профиля от списка ссылок. Весь компонент появляется на странице при загрузке посредством CSS-анимации fadeDown, плавно перемещающей его из отрицательного смещения по оси Y к нулевому с одновременным нарастанием прозрачности.

3.6 Разработка компонента карточки ссылки

Компонент LinkCard.vue является наиболее интерактивным элементом приложения и сосредотачивает в себе большую часть визуальной выразительности интерфейса. Компонент принимает два входных параметра: объект link, содержащий все данные ссылки, и числовое значение delay, определяющее задержку анимации появления карточки при загрузке страницы.

Корневым элементом компонента является элемент якорной ссылки a с атрибутами target="_blank" и rel="noopener noreferrer". Использование семантически корректного элемента ссылки вместо элемента div с обработчиком события click обеспечивает

корректную работу вспомогательных технологий, поддержку открытия ссылки в новой вкладке через контекстное меню браузера и правильное поведение при навигации с клавиатуры. Атрибут rel со значением noopener noreferrer является мерой безопасности, предотвращающей доступ открытой страницы к объекту window открывшей её страницы.

Для отслеживания состояния наведения курсора в компоненте объявляется реактивная переменная hovered посредством функции ref с начальным значением false. Обработчики событий mouseenter и mouseleave, привязанные к корневому элементу, обновляют значение переменной. Переменная hovered используется исключительно для условного применения класса card__arrow--active к элементу стрелки — все остальные эффекты наведения реализованы средствами CSS посредством псевдокласса :hover, что является более производительным подходом по сравнению с управлением стилями через JavaScript.

Система цветовых акцентов построена на механизме CSS-переменных и классов-модификаторов. К корневому элементу карточки динамически добавляется класс вида card--pink, card--cyan, card--purple или card--yellow в зависимости от значения поля color объекта link. Каждый класс-модификатор переопределяет значение локальной CSS-переменной --c соответствующим акцентным цветом. Все цветозависимые свойства карточки ссылаются на переменную --c, что обеспечивает полную консистентность цветового акцента во всех визуальных состояниях элемента без дублирования кода.

Задержка анимации появления передаётся через входной параметр delay и применяется к корневому элементу карточки посредством инлайнового стиля animation-delay с вычисленным значением в миллисекундах. Это позволяет компоненту LinkList управлять порядком появления карточек, передавая каждой карточке индивидуальное значение задержки, вычисленное на основе её порядкового номера в списке.

Световой блик при наведении реализован абсолютно позиционированным элементом с линейным градиентом, в обычном состоянии смещённым за левый край карточки. При наведении CSS-переход плавно перемещает элемент за правый край, создавая эффект скользящего светового блика. Глитч-эффект реализован абсолютно позиционированным элементом, дублирующим текст метки карточки и воспроизводящим при наведении кратковременную анимацию с изменением свойств clip-path и transform в ступенчатом режиме steps(2), имитируя помехи видеосигнала. Элемент стрелки плавно появляется из левой стороны при наведении посредством CSS-перехода свойств opacity и transform.

3.7 Разработка компонента списка ссылок

Компонент LinkList.vue реализован максимально лаконично в соответствии с принципом единственной ответственности. Его единственная задача состоит в рендеринге переданного массива ссылок в виде последовательности карточек с индивидуальными задержками анимации появления. Компонент принимает один входной параметр links — массив объектов ссылок — и объявляет его посредством макроса defineProps с указанием типа Array и обязательности.

Рендеринг списка выполняется посредством директивы v-for, итерирующей по массиву links с одновременным извлечением элемента и его порядкового индекса. В качестве значения обязательного атрибута key используется поле id объекта ссылки при его наличии или порядковый индекс в качестве запасного варианта. Атрибут key необходим Vue для эффективного обновления DOM посредством алгоритма виртуального DOM: при наличии уникального ключа Vue способен точно определить, какие узлы дерева требуют обновления при изменении данных, вместо полного перерендеринга списка.

Значение задержки анимации для каждой карточки вычисляется непосредственно в шаблоне как произведение порядкового индекса элемента и константы 80 миллисекунд. Таким образом обеспечивается каскадный эффект появления карточек при загрузке страницы: первая карточка появляется без задержки, каждая последующая — на 80 миллисекунд позже предыдущей. Блок стилей компонента содержит минимум кода: объявление flex-контейнера с вертикальным направлением и отступом между карточками в 12 пикселей.

3.8 Интеграция компонентов и передача данных

После разработки всех пяти компонентов и конфигурационного файла был выполнен этап интеграции — проверки корректности взаимодействия компонентов в составе целого приложения. Интеграция в данном контексте означает не написание дополнительного кода, а проверку того, что поток данных от конфигурационного файла через корневой компонент к листовым компонентам дерева работает корректно и все компоненты отображают ожидаемый результат.

В ходе интеграционной проверки последовательно тестировались следующие сценарии. Проверялось, что изменение значения поля name в объекте profile конфигурационного файла немедленно отражается в заголовке компонента ProfileHeader в браузере в режиме горячей замены модулей без перезагрузки страницы. Проверялось, что добавление нового объекта в массив links приводит к появлению соответствующей карточки в списке с корректным цветовым акцентом и иконкой. Проверялось, что удаление объекта из массива приводит к исчезновению соответствующей карточки без нарушения каскадной анимации появления оставшихся карточек.

Отдельно проверялась корректность передачи значения задержки анимации от компонента LinkList к компоненту LinkCard. Для этого в массив links временно добавлялось большое количество объектов, что позволило визуально убедиться в том, что каждая карточка появляется с нарастающей задержкой и каскадный эффект работает корректно при любом количестве элементов.

Механизм горячей замены модулей Vite, обновляющий изменённые компоненты в браузере без полной перезагрузки страницы и без потери текущего состояния приложения, существенно ускорил процесс интеграционной проверки. Любое изменение в коде компонента или конфигурационном файле отражалось в браузере в течение долей секунды, что позволяло немедленно наблюдать результат каждого вносимого изменения.

3.9 Обеспечение адаптивности интерфейса

Адаптивность приложения обеспечивается совокупностью нескольких независимых подходов, применённых на уровне различных компонентов. Принципиальным решением является отказ от фиксированных пиксельных значений там, где это возможно, в пользу относительных единиц и современных CSS-функций.

Основной контейнер страницы имеет ограничение максимальной ширины в 520 пикселей, что обеспечивает оптимальную читаемость содержимого на широкоформатных экранах. При ширине области просмотра меньше этого значения контейнер занимает всю доступную ширину с симметричными горизонтальными отступами в 20 пикселей, предотвращая прилипание содержимого к краям экрана на мобильных устройствах. Данное поведение достигается без медиа-запросов: свойство max-width в сочетании со значением width: 100% автоматически обеспечивает описанное поведение на любой ширине.

Размер шрифта заголовка имени пользователя в компоненте ProfileHeader задан посредством функции clamp(20px, 5vw, 26px). Это означает, что размер шрифта никогда не опустится ниже 20 пикселей и не превысит 26 пикселей, плавно масштабируясь между этими значениями пропорционально ширине области просмотра. Данный подход обеспечивает читаемость заголовка на узких экранах мобильных устройств и предотвращает излишнее увеличение на широких экранах.

Компонент BackgroundEffect корректно обрабатывает изменение размера окна браузера посредством обработчика события resize, привязанного к объекту window при монтировании компонента. При срабатывании обработчика размеры холста элемента canvas обновляются до актуальных значений ширины и высоты окна. Поскольку координаты частиц хранятся в нормализованном виде, изменение размера холста не нарушает их относительного расположения.

Карточки ссылок в компоненте LinkCard не имеют фиксированной высоты — она определяется содержимым и внутренними отступами. Это обеспечивает корректное отображение карточек при любой длине текста метки без риска переполнения или обрезки содержимого. Иконки карточек имеют фиксированный размер и не масштабируются, поскольку их размер в 20 пикселей является оптимальным для восприятия на любом устройстве.

3.10 Тестирование приложения

После завершения разработки всех компонентов было проведено функциональное тестирование приложения в браузере. Тестирование выполнялось в режиме сервера разработки Vite, обеспечивающего мгновенное применение изменений и удобный доступ к инструментам разработчика браузера.

Тестирование корректности отображения профиля включало проверку поведения при заполненном и пустом значении поля avatar. При заполненном значении проверялось корректное отображение изображения внутри вращающегося кольца. При пустом значении проверялось автоматическое формирование и отображение инициалов с

градиентным оформлением. Дополнительно проверялось поведение при различных значениях поля bio: многострочный текст с переносом строки отображался корректно, сохраняя структуру.

Тестирование карточек ссылок включало проверку всех четырёх цветовых акцентов, обоих форматов иконок и корректности переходов по ссылкам. Для каждого цветового акцента проверялось соответствие цвета свечения при наведении заявленному акцентному цвету палитры. Переходы по ссылкам тестировались путём нажатия на каждую карточку с проверкой открытия целевого адреса в новой вкладке браузера. Ссылка формата mailto тестировалась отдельно: при нажатии должна открываться программа электронной почты по умолчанию или соответствующий веб-интерфейс.

Тестирование анимационных эффектов включало визуальную проверку всех реализованных анимаций: вращения кольца аватара, движения перспективной сетки, плавания частиц на холсте, каскадного появления карточек при загрузке страницы, светового блика и глитч-эффекта при наведении. Все анимации воспроизводились корректно без визуальных сбоев и артефактов.

Тестирование адаптивности выполнялось в инструментах разработчика браузера посредством режима эмуляции устройств. Проверялись ширины области просмотра 320, 375, 768 и 1440 пикселей, соответствующие минимальной ширине мобильного устройства, стандартной ширине смартфона, ширине планшета и ширине настольного монитора. При всех проверяемых значениях ширины интерфейс отображался корректно без горизонтальной прокрутки и без нарушения читаемости текста.

3.11 Сборка и подготовка к развёртыванию

Завершающим этапом практической части проекта стала производственная сборка приложения и подготовка его к развёртыванию на хостинге. Производственная сборка выполнялась командой npm run build, запускающей инструмент сборки Vite в режиме оптимизации.

В ходе сборки Vite выполнил следующие операции: статический анализ графа зависимостей модулей для определения кода, включаемого в итоговый бандл; разделение кода на несколько файлов-чанков для оптимизации загрузки; минификацию JavaScript-кода посредством инструмента esbuild, удаляющего пробелы, комментарии и сокращающего имена переменных; минификацию CSS-кода с удалением избыточных правил и пробелов; формирование итоговых файлов с хэшами в именах для обеспечения корректного кэширования браузером.

Результатом сборки стала директория dist, содержащая три файла: index.html с обновлёнными ссылками на ресурсы, минифицированный JavaScript-файл и минифицированный CSS-файл.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики по профессиональному модулю ПМ3 «Разработка модулей программного обеспечения для компьютерных систем» авторами проекта было спроектировано и разработано веб-приложение «Link Showcase» — персональный конструктор витрины ссылок с уникальным визуальным оформлением в эстетике синтвейв.

Все задачи, сформулированные во введении к отчёту, были выполнены в полном объёме. Проведён анализ существующих аналогов в категории персональных витрин ссылок, по результатам которого выявлены ключевые недостатки коммерческих решений и сформулированы требования к разрабатываемому приложению. Обоснован выбор технологического стека на основе Vue 3 и Vite. Спроектирована компонентная архитектура приложения, включающая пять однофайловых компонентов и конфигурационный файл данных. Реализован выразительный анимированный интерфейс с многоуровневым фоновым эффектом, интерактивными карточками ссылок и плавными анимациями появления. Обеспечена возможность полной настройки содержимого страницы посредством редактирования единственного конфигурационного файла. Проведено функциональное тестирование всех реализованных возможностей и выполнена производственная сборка проекта.

В процессе выполнения практики были приобретены и закреплены следующие профессиональные навыки и компетенции. В области проектирования — навыки декомпозиции пользовательского интерфейса на компоненты с применением принципа единственной ответственности, проектирования структуры данных клиентского приложения и планирования потока данных в дереве компонентов. В области разработки — практические навыки работы с Composition API Vue 3, синтаксисом script setup, реактивными переменными ref и вычисляемыми свойствами computed, хуками жизненного цикла onMounted и onUnmounted, а также директивами шаблона v-for, v-if и v-else. В области стилизации — навыки применения CSS-переменных для построения цветовых систем, создания сложных анимационных эффектов средствами нативного CSS, реализации адаптивной типографики посредством функции clamp и обеспечения изоляции стилей компонентов посредством атрибута scoped. В области работы с браузерными API — навыки работы с Canvas API для реализации процедурной анимации, управления циклом requestAnimationFrame и корректного освобождения ресурсов при размонтировании компонентов.

Разработанное приложение полностью соответствует всем критериям готовности, сформулированным в разделе 1.5 настоящего отчёта. Все функциональные и нефункциональные требования реализованы и проверены в ходе тестирования. Производственная сборка завершается успешно и формирует корректный статический бандл, готовый к немедленному развёртыванию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Vue.js. (2024). Vue 3 — официальная документация. Vue.js. https://vuejs.org/guide/introduction

2 Vite. (2024). Vite — официальная документация. Vite. https://vitejs.dev/guide/

3 Mozilla Developer Network. (2024). Web APIs: Canvas API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

4 Mozilla Developer Network. (2024). CSS: Custom properties (variables). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

5 Mozilla Developer Network. (2024). CSS animations. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_animations

6 Mozilla Developer Network. (2024). requestAnimationFrame. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame

7 Vue.js. (2024). Composition API FAQ. Vue.js. https://vuejs.org/guide/extras/composition-api-faq

8 Vue.js. (2024). Single File Components. Vue.js. https://vuejs.org/guide/scaling-up/sfc

9 Google Fonts. (2024). Orbitron font family. Google Fonts. https://fonts.google.com/specimen/Orbitron

10 Google Fonts. (2024). Share Tech Mono font family. Google Fonts. https://fonts.google.com/specimen/Share+Tech+Mono

11 Mozilla Developer Network. (2024). CSS: clamp() function. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/clamp

12 Mozilla Developer Network. (2024). CSS: backdrop-filter. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter

13 Linktree. (2024). Официальный сайт сервиса Linktree. Linktree. https://linktr.ee

14 Taplink. (2024). Официальный сайт сервиса Taplink. Taplink. https://taplink.at

15 Evans, Y. (2023). Announcing Vite 5.0. Vite Blog. https://vitejs.dev/blog/announcing-vite5